

CST-407 Activity 3 Password Hashing and Cracking

Alex M. Frear

College of Science, Engineering, and Technology, Grand Canyon University

Course Number: CST-407

Professor: Dr. Melody White

08/08/2025

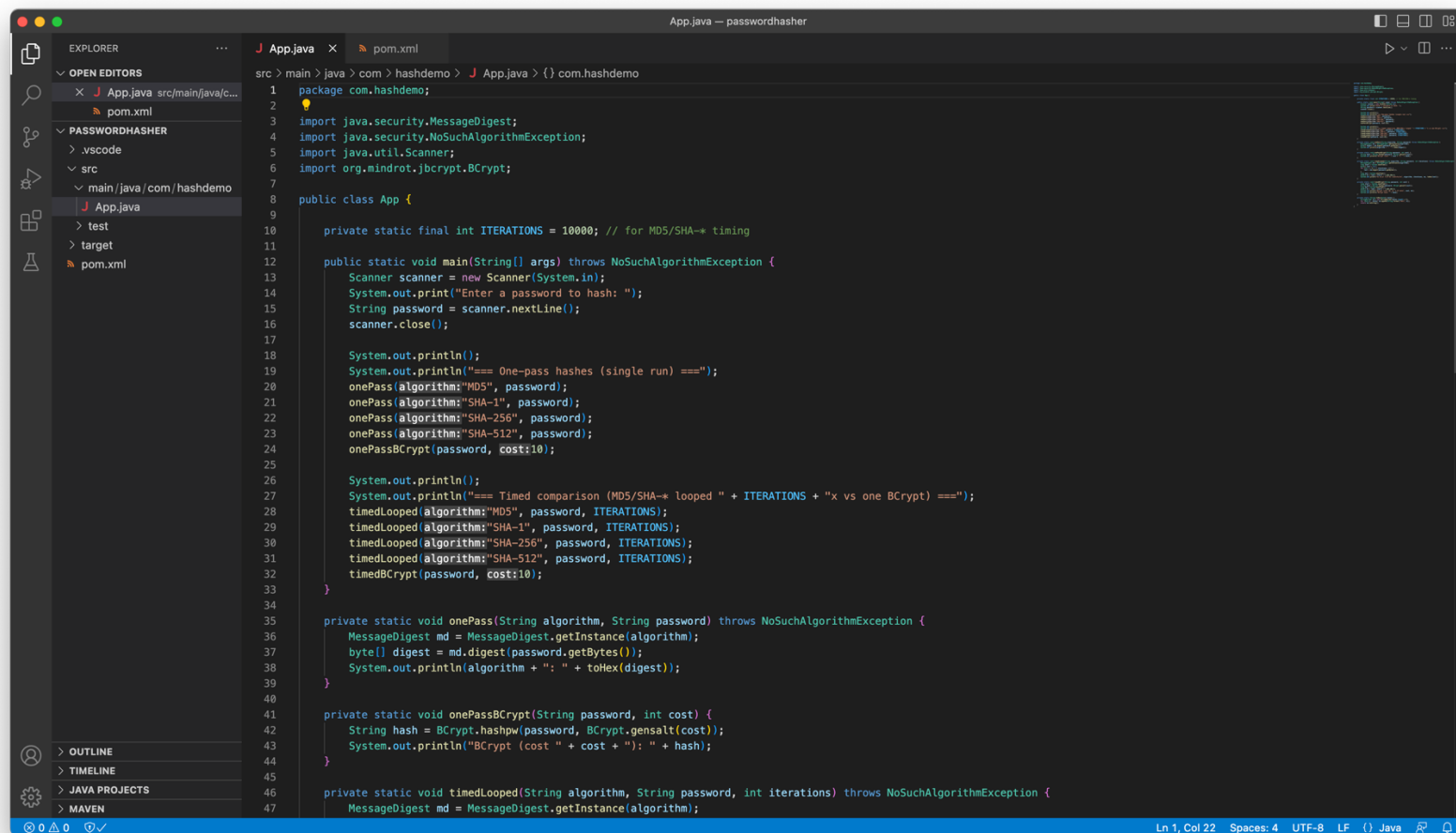
Table of Contents

Part 2: Write a Java Program That Hashes a Password	3
Screenshots	3
App.java Source Code	3
App.java Hashed Password Output.....	5
Part 3: Cracking the Hashed Password	6
Screenshots	6
Successful Dictionary Attack Result	6
App.java Password Input Prompt.....	7
App Hashing Prompt and MD5 Output	8
Selecting Brute-force Alphabet	9
Successful Brute-Force Attack Result	10
Part 4: Toy Blockchain.....	11
Screenshots	11
Blockchain Program Output	11
Summary of Key Concepts.....	12
Password Cracking Time Estimates and Best Practices	13
Password Cracking Time Estimate Table	13
Best Practices for Creating Secure Passwords.....	14

Part 2: Write a Java Program That Hashes a Password

Screenshots

App.java Source Code



The screenshot shows a VS Code editor window titled "App.java — passwordhasher". The Explorer sidebar on the left shows the project structure: "src/main/java/com/hashdemo" containing "App.java" and "pom.xml". The main editor displays the source code for "App.java".

```
src > main > java > com > hashdemo > J App.java > {} com.hashdemo
1 package com.hashdemo;
2
3 import java.security.MessageDigest;
4 import java.security.NoSuchAlgorithmException;
5 import java.util.Scanner;
6 import org.mindrot.jbcrypt.BCrypt;
7
8 public class App {
9
10     private static final int ITERATIONS = 10000; // for MD5/SHA-> timing
11
12     public static void main(String[] args) throws NoSuchAlgorithmException {
13         Scanner scanner = new Scanner(System.in);
14         System.out.print("Enter a password to hash: ");
15         String password = scanner.nextLine();
16         scanner.close();
17
18         System.out.println();
19         System.out.println("=== One-pass hashes (single run) ===");
20         onePass(algorithm:"MD5", password);
21         onePass(algorithm:"SHA-1", password);
22         onePass(algorithm:"SHA-256", password);
23         onePass(algorithm:"SHA-512", password);
24         onePassBCrypt(password, cost:10);
25
26         System.out.println();
27         System.out.println("=== Timed comparison (MD5/SHA-> looped " + ITERATIONS + "x vs one BCrypt) ===");
28         timedLooped(algorithm:"MD5", password, ITERATIONS);
29         timedLooped(algorithm:"SHA-1", password, ITERATIONS);
30         timedLooped(algorithm:"SHA-256", password, ITERATIONS);
31         timedLooped(algorithm:"SHA-512", password, ITERATIONS);
32         timedBCrypt(password, cost:10);
33     }
34
35     private static void onePass(String algorithm, String password) throws NoSuchAlgorithmException {
36         MessageDigest md = MessageDigest.getInstance(algorithm);
37         byte[] digest = md.digest(password.getBytes());
38         System.out.println(algorithm + ": " + toHex(digest));
39     }
40
41     private static void onePassBCrypt(String password, int cost) {
42         String hash = BCrypt.hashpw(password, BCrypt.gensalt(cost));
43         System.out.println("BCrypt (cost " + cost + "): " + hash);
44     }
45
46     private static void timedLooped(String algorithm, String password, int iterations) throws NoSuchAlgorithmException {
47         MessageDigest md = MessageDigest.getInstance(algorithm);
```

The status bar at the bottom indicates "Ln 1, Col 22", "Spaces: 4", "UTF-8", "LF", "Java", and a search icon.

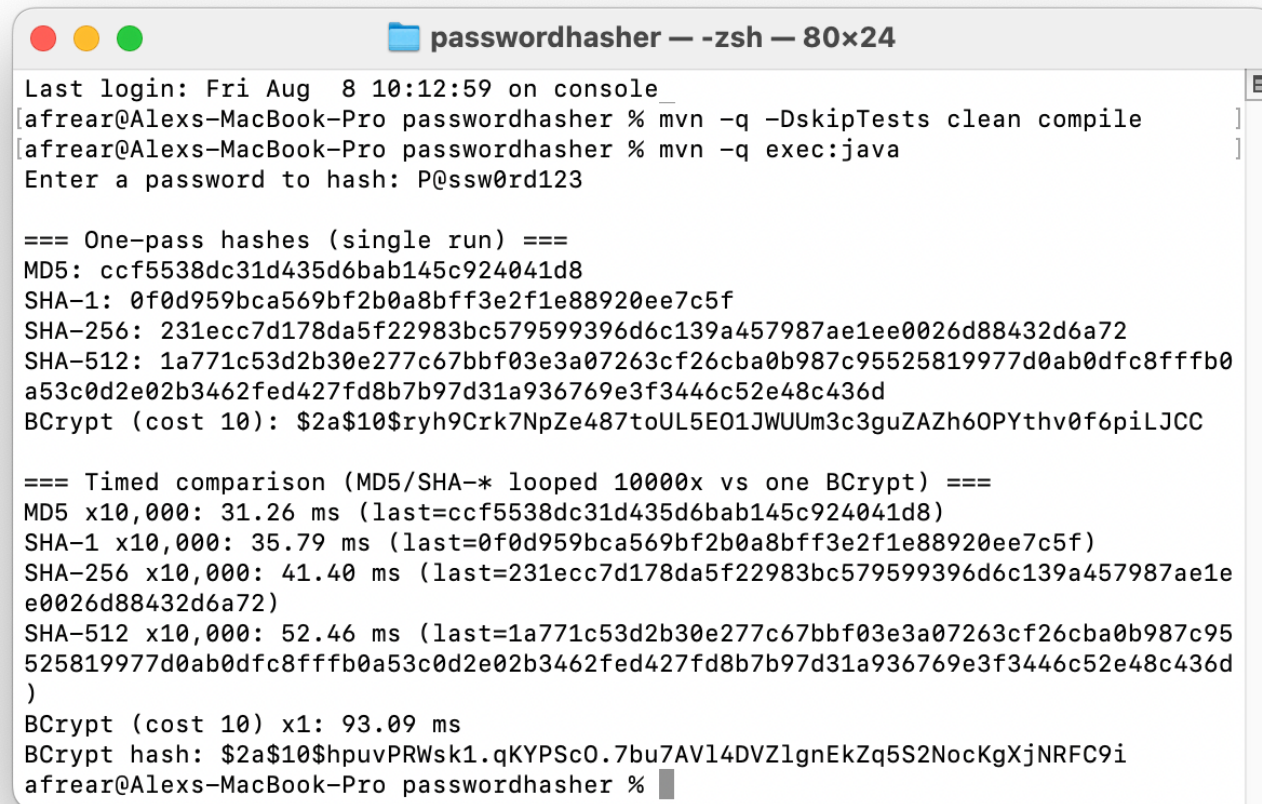
```
App.java — passwordhasher

src > main > java > com > hashdemo > App.java > {} com.hashdemo

46 private static void timedLooped(String algorithm, String password, int iterations) throws NoSuchAlgorithmException {
47     MessageDigest md = MessageDigest.getInstance(algorithm);
48     long start = System.nanoTime();
49     byte[] last = null;
50     for (int i = 0; i < iterations; i++) {
51         last = md.digest(password.getBytes());
52     }
53     long end = System.nanoTime();
54     double ms = (end - start) / 1_000_000.0;
55     System.out.printf("%s x%d: %.2f ms (last=%s)\n", algorithm, iterations, ms, toHex(last));
56 }
57
58 private static void timedBCrypt(String password, int cost) {
59     long start = System.nanoTime();
60     String hash = BCrypt.hashpw(password, BCrypt.gensalt(cost));
61     long end = System.nanoTime();
62     double ms = (end - start) / 1_000_000.0;
63     System.out.printf("BCrypt (cost %d) x1: %.2f ms\n", cost, ms);
64     System.out.println("BCrypt hash: " + hash);
65 }
66
67 private static String toHex(byte[] bytes) {
68     StringBuilder sb = new StringBuilder(bytes.length * 2);
69     for (byte b : bytes) sb.append(String.format("%02x", b));
70     return sb.toString();
71 }
72
73 }
```

Figure 1 Java password hashing program implementing MD5, SHA-1, SHA-256, SHA-512, and BCrypt, with timing comparisons to illustrate BCrypt's computational cost.

App.java Hashed Password Output



```
passwordhasher — -zsh — 80x24

Last login: Fri Aug  8 10:12:59 on console_
[afrear@Alexs-MacBook-Pro passwordhasher % mvn -q -DskipTests clean compile ]
[afrear@Alexs-MacBook-Pro passwordhasher % mvn -q exec:java ]
Enter a password to hash: P@ssw0rd123

=== One-pass hashes (single run) ===
MD5: ccf5538dc31d435d6bab145c924041d8
SHA-1: 0f0d959bca569bf2b0a8bff3e2f1e88920ee7c5f
SHA-256: 231ecc7d178da5f22983bc579599396d6c139a457987ae1ee0026d88432d6a72
SHA-512: 1a771c53d2b30e277c67bbf03e3a07263cf26cba0b987c95525819977d0ab0dfc8fffb0
a53c0d2e02b3462fed427fd8b7b97d31a936769e3f3446c52e48c436d
BCrypt (cost 10): $2a$10$ryh9Crk7NpZe487toUL5E01JWUUm3c3guZAZh6OPYthv0f6piLJCC

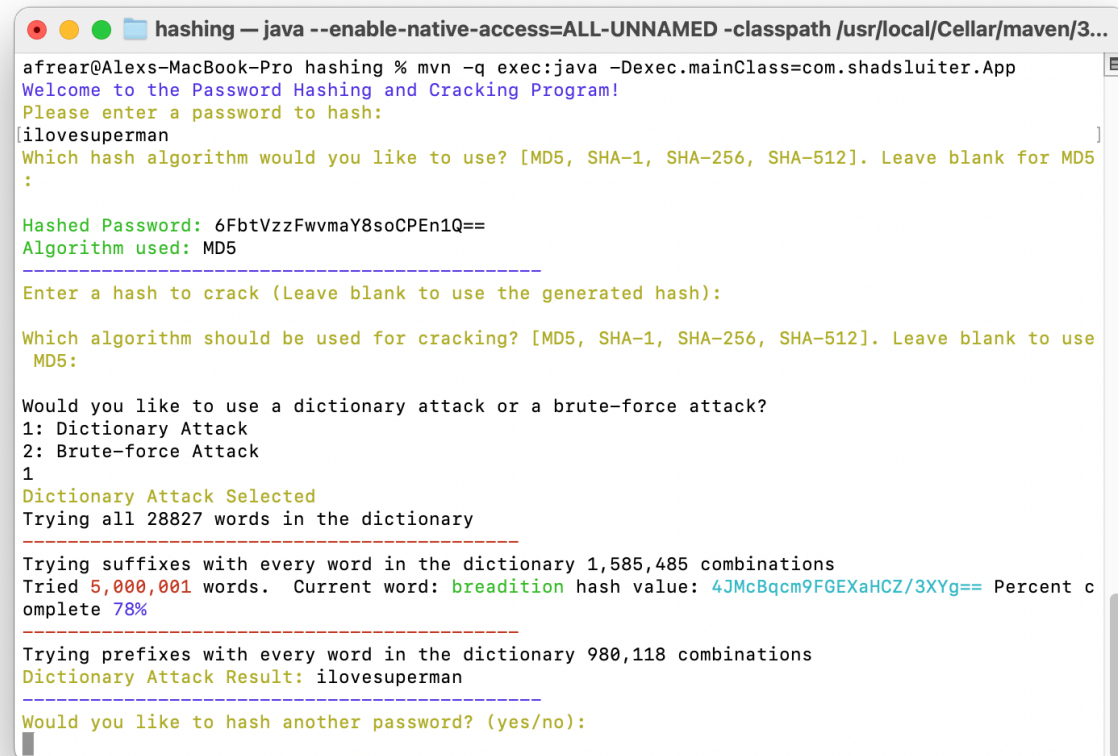
=== Timed comparison (MD5/SHA-* looped 10000x vs one BCrypt) ===
MD5 x10,000: 31.26 ms (last=ccf5538dc31d435d6bab145c924041d8)
SHA-1 x10,000: 35.79 ms (last=0f0d959bca569bf2b0a8bff3e2f1e88920ee7c5f)
SHA-256 x10,000: 41.40 ms (last=231ecc7d178da5f22983bc579599396d6c139a457987ae1e
e0026d88432d6a72)
SHA-512 x10,000: 52.46 ms (last=1a771c53d2b30e277c67bbf03e3a07263cf26cba0b987c95
525819977d0ab0dfc8fffb0a53c0d2e02b3462fed427fd8b7b97d31a936769e3f3446c52e48c436d
)
BCrypt (cost 10) x1: 93.09 ms
BCrypt hash: $2a$10$hpuvPRWsk1.qKYPSc0.7bu7AVl4DVZlgnEkZq5S2NocKgXjNRFC9i
afrear@Alexs-MacBook-Pro passwordhasher %
```

Figure 2 Hashing the same password with MD5, SHA-1, SHA-256, SHA-512, and BCrypt, plus timing comparison for 10,000 iterations of MD5/SHA- versus one BCrypt hash (cost 10). BCrypt is intentionally slower to resist brute-force attacks.

Part 3: Cracking the Hashed Password

Screenshots

Successful Dictionary Attack Result



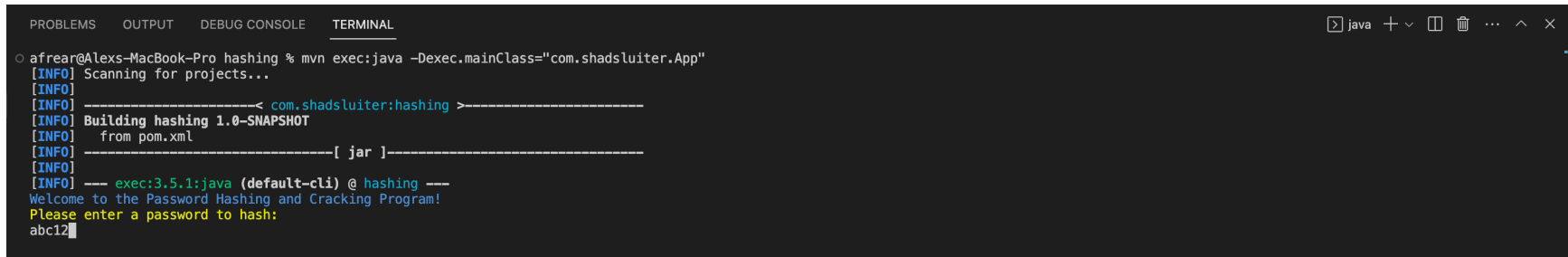
```
afrear@Alexs-MacBook-Pro hashing % mvn -q exec:java -Dexec.mainClass=com.shadsluiter.App
Welcome to the Password Hashing and Cracking Program!
Please enter a password to hash:
[ilovesuperman]
Which hash algorithm would you like to use? [MD5, SHA-1, SHA-256, SHA-512]. Leave blank for MD5
:
Hashed Password: 6FbtVzzFwvmaY8soCPEn1Q==
Algorithm used: MD5
-----
Enter a hash to crack (Leave blank to use the generated hash):

Which algorithm should be used for cracking? [MD5, SHA-1, SHA-256, SHA-512]. Leave blank to use
MD5:

Would you like to use a dictionary attack or a brute-force attack?
1: Dictionary Attack
2: Brute-force Attack
1
Dictionary Attack Selected
Trying all 28827 words in the dictionary
-----
Trying suffixes with every word in the dictionary 1,585,485 combinations
Tried 5,000,001 words. Current word: breadition hash value: 4JMcBqcm9FGEXaHCZ/3XYg== Percent c
omplete 78%
-----
Trying prefixes with every word in the dictionary 980,118 combinations
Dictionary Attack Result: ilovesuperman
-----
Would you like to hash another password? (yes/no):
```

Figure 3 Dictionary attack on an MD5 hash of “ilovesuperman.” The program searched the provided wordlist and rapidly recovered in seconds the original password, demonstrating why dictionary words are insecure for password creation.

App.java Password Input Prompt



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
afrear@Alexs-MacBook-Pro hashing % mvn exec:java -Dexec.mainClass="com.shadsluiter.App"
[INFO] Scanning for projects...
[INFO] -----< com.shadsluiter:hashing >-----
[INFO] Building hashing 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- exec:3.5.1:java (default-cli) @ hashing ---
Welcome to the Password Hashing and Cracking Program!
Please enter a password to hash:
abc12
```

Figure 4 The user entered a new password "abc12" to generate a hash using the Password Hashing and Cracking Program.

App Hashing Prompt and MD5 Output

```
Welcome to the Password Hashing and Cracking Program!  
Please enter a password to hash:  
abc12  
Which hash algorithm would you like to use? [MD5, SHA-1, SHA-256, SHA-512]. Leave blank for MD5:  
  
Hashed Password: shV+eyrnFqdHwXCx8e+3oA==  
Algorithm used: MD5  
-----  
Enter a hash to crack (Leave blank to use the generated hash):
```

Figure 5 User enters the password abc12 and selects MD5 as the hashing algorithm. The application returns the hashed value shV+eyrnFqdHwXCx8e+3oA== using MD5.

Selecting Brute-force Alphabet

```
Welcome to the Password Hashing and Cracking Program!
Please enter a password to hash:
abc12
Which hash algorithm would you like to use? [MD5, SHA-1, SHA-256, SHA-512]. Leave blank for MD5:

Hashed Password: shV+eyrnFqdHwXcX8e+3oA==
Algorithm used: MD5

Enter a hash to crack (Leave blank to use the generated hash):

Which algorithm should be used for cracking? [MD5, SHA-1, SHA-256, SHA-512]. Leave blank to use MD5:

Would you like to use a dictionary attack or a brute-force attack?
1: Dictionary Attack
2: Brute-force Attack
2
Brute-force Attack Selected
Select an alphabet for brute-force attack (Enter 0-14, Leave blank for 0):
0: abcdefghijklmnopqrstuvwxyz
1: ABCDEFGHIJKLMNOPQRSTUVWXYZ
2: 0123456789
3: !@#$%^&*()-_+=[]{}|;:,.<>?
4: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
5: abcdefghijklmnopqrstuvwxyz0123456789
6: abcdefghijklmnopqrstuvwxyz!@#$%^&*()-_+=[]{}|;:,.<>?
7: ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789
8: ABCDEFGHIJKLMNOPQRSTUVWXYZ!@#$%^&*()-_+=[]{}|;:,.<>?
9: 0123456789!@#$%^&*()-_+=[]{}|;:,.<>?
10: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789
11: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!@#$%^&*()-_+=[]{}|;:,.<>?
12: abcdefghijklmnopqrstuvwxyz0123456789!@#$%^&*()-_+=[]{}|;:,.<>?
13: ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#$%^&*()-_+=[]{}|;:,.<>?
14: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#$%^&*()-_+=[]{}|;:,.<>?
5
```

Figure 6 User selects a brute-force attack and chooses option 5 (lowercase letters + numbers) for the character set, which matches the format of the password abc12.

Successful Brute-Force Attack Result

```
Welcome to the Password Hashing and Cracking Program!
Please enter a password to hash:
abc12
Which hash algorithm would you like to use? [MD5, SHA-1, SHA-256, SHA-512]. Leave blank for MD5:

Hashed Password: shV+eyrnFqdHwXcX8e+3oA==
Algorithm used: MD5

Enter a hash to crack (Leave blank to use the generated hash):

Which algorithm should be used for cracking? [MD5, SHA-1, SHA-256, SHA-512]. Leave blank to use MD5:

Would you like to use a dictionary attack or a brute-force attack?
1: Dictionary Attack
2: Brute-force Attack
2
Brute-force Attack Selected
Select an alphabet for brute-force attack (Enter 0-14, Leave blank for 0):
0: abcdefghijklmnopqrstuvwxyz
1: ABCDEFGHIJKLMNOPQRSTUVWXYZ
2: 0123456789
3: !@#$%^&*()-_+=[]{}|;:,.<>?
4: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
5: abcdefghijklmnopqrstuvwxyz0123456789
6: abcdefghijklmnopqrstuvwxyz!@#$%^&*()-_+=[]{}|;:,.<>?
7: ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789
8: ABCDEFGHIJKLMNOPQRSTUVWXYZ!@#$%^&*()-_+=[]{}|;:,.<>?
9: 0123456789!@#$%^&*()-_+=[]{}|;:,.<>?
10: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789
11: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!@#$%^&*()-_+=[]{}|;:,.<>?
12: abcdefghijklmnopqrstuvwxyz0123456789!@#$%^&*()-_+=[]{}|;:,.<>?
13: ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#$%^&*()-_+=[]{}|;:,.<>?
14: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#$%^&*()-_+=[]{}|;:,.<>?
5

Trying length: 1 which will have 36 combinations
Trying length: 2 which will have 1,296 combinations
Trying length: 3 which will have 46,656 combinations
Trying length: 4 which will have 1,679,616 combinations
Trying length: 5 which will have 60,466,176 combinations
Brute Force Attack Result: abc12 is the password. shV+eyrnFqdHwXcX8e+3oA== is the hash value. Which was hashed with MD5

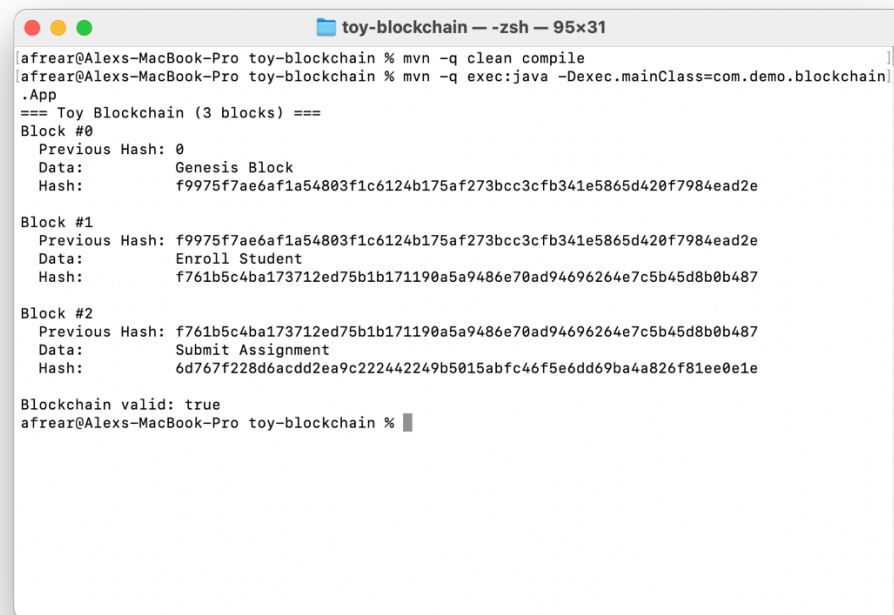
Would you like to hash another password? (yes/no):
```

Figure 7 The program attempts combinations from length 1 through 5 using the selected character set (lowercase + numbers). The brute-force attack successfully cracks the MD5 hash and reveals the original password abc12. The hash value shV+eyrnFqdHwXcX8e+3oA== matches the one generated during hashing.

Part 4: Toy Blockchain

Screenshots

Blockchain Program Output



```
afrear@Alexs-MacBook-Pro toy-blockchain % mvn -q clean compile
afrear@Alexs-MacBook-Pro toy-blockchain % mvn -q exec:java -Dexec.mainClass=com.demo.blockchain
.App
=== Toy Blockchain (3 blocks) ===
Block #0
  Previous Hash: 0
  Data: Genesis Block
  Hash: f9975f7ae6af1a54803f1c6124b175af273bcc3cfb341e5865d420f7984ead2e
Block #1
  Previous Hash: f9975f7ae6af1a54803f1c6124b175af273bcc3cfb341e5865d420f7984ead2e
  Data: Enroll Student
  Hash: f761b5c4ba173712ed75b1b171190a5a9486e70ad94696264e7c5b45d8b0b487
Block #2
  Previous Hash: f761b5c4ba173712ed75b1b171190a5a9486e70ad94696264e7c5b45d8b0b487
  Data: Submit Assignment
  Hash: 6d767f228d6acdd2ea9c222442249b5015abfc46f5e6dd69ba4a826f81ee0e1e
Blockchain valid: true
afrear@Alexs-MacBook-Pro toy-blockchain %
```

Figure 8 This toy blockchain demonstrates how each block stores application data, a timestamp, its own SHA-256 hash, and the previous block's hash, linking them together into an immutable sequence. Unlike a production blockchain, this example runs on a single node and lacks distributed consensus (no Proof-of-Work or Proof-of-Stake), peer-to-peer networking, digital signatures, transaction handling, difficulty adjustments, or incentives. It serves as an educational model to show how cryptographic hashing preserves block integrity and detects tampering.

Summary of Key Concepts

This was my first time ever trying password hashing and cracking, and it was honestly a lot more eye-opening than I expected. I had heard of brute-force attacks before, mostly in general conversations about cybersecurity, but I didn't really know how they worked in practice. I also wasn't familiar with concepts like hashing algorithms (MD5, SHA-1, etc.) or how hashed passwords are actually stored and protected. Going through this activity gave me a hands-on look at how these systems work and why they're important.

Earlier in the activity, I tested a longer password (Hunter2025) just out of curiosity, and the cracking process ran for hours without success. In contrast, the final test I documented used a much shorter password (abc12), which was cracked in a matter of seconds. That difference really stuck with me. It showed me just how much a few extra characters or the use of upper/lowercase and symbols can change how secure a password is. It was a good lesson in why password complexity matters.

This experience helped me finally understand why cybersecurity experts always emphasize using strong passwords and why certain hashing algorithms like MD5 are no longer considered secure. Now that I've actually run a brute-force attack myself, I have a much better appreciation for both the risks and the protections involved in password security. This was a solid introduction to a topic I previously only knew by name.

Password Cracking Time Estimates and Best Practices

Password Cracking Time Estimate Table

Password Type	Example	Cracking Time Estimate	Notes
6 lowercase letters	kitten	Less than 1 second	Easily cracked with brute-force
8 lowercase letters	sunshine	~2 minutes	Slightly better but still weak
8 mixed-case letters	Summer19	~22 minutes	Adds complexity but still vulnerable
8 chars w/ letters & numbers	abc12345	~6 minutes	Common format; easily guessed
10 chars w/ mixed-case + num	P@ssw0rd21	~3 days	Better but predictable patterns can be cracked
12 chars w/ symbols	Tr!ckyP@ss2025	~34 years	Strong if not based on dictionary words
16 random chars (full set)	x7@B4#zP!9qT0\$Im	Trillions of years	Considered secure against brute-force

Best Practices for Creating Secure Passwords

This was my first time working with password hashing and cracking, and the experience was eye-opening. I originally used a stronger password like “Hunter2025,” which resisted brute-force attempts even after more than 16 billion guesses. Later, I tried a simpler password like “abc12,” and it was cracked almost instantly.

Seeing how quickly weak passwords fall to brute-force attacks helped me understand why strong passwords are so critical. Even small improvements, like adding symbols or increasing length, drastically improve a password’s strength.

Based on what I learned, here are a few best practices to follow:

- Use **12+ characters** with uppercase, lowercase, numbers, and special characters.
- Avoid common words, names, or obvious patterns like “123” or “password.”
- Use **passphrases** that are long but memorable (e.g., Ocean\$Blue!Horse47).
- Use a **password manager** to generate and store complex passwords securely.
- Never reuse the same password across multiple websites or apps.

These practices can make brute-force or dictionary-based attacks much less effective, helping keep your accounts and information safe.